



MANUEL D'UTILISATION DU SIMULATEUR

Apprentissage par renforcement d'une politique de navigation mono-agent avec l'algorithme SAC

SUJET DU STAGE : Étude d'algorithmes d'apprentissage par renforcement pour l'optimisation en temps du transport des bagages par des véhicules guidés automatisés (AGV) en environnement aéroportuaire.

AUTEURE : Sayan Suos

Table des matières

Introduction	2
1 Installation du simulateur	3
1.1 Prérequis	3
1.2 Installation	3
2 Structure du projet	4
2.1 Organisation des fichiers	4
2.2 Architecture du simulateur	4
2.3 Organisation des classes	5
2.4 Principaux composants du simulateur	6
3 Configuration du simulateur	7
4 Utilisation du simulateur	8
5 Personnalisation du simulateur	9
6 Résultats générés	10

Introduction

To do : Objectif du simulateur – Fonctionnalités principales – Architecture générale – Organisation du document

1 Installation du simulateur

Cette section décrit les différentes étapes nécessaires pour installer le simulateur sur une machine locale. Les procédures présentées ont été testées sous macOS et Windows.

1.1 Prérequis

Avant d'installer le simulateur, les logiciels suivants doivent être installés sur la machine :

- Python (version 3.11 ou supérieure) ;
- Git, utilisé pour cloner le dépôt du projet ;
- un environnement virtuel Python, permettant d'isoler les dépendances du projet des autres installations Python présentes sur la machine.

1.2 Installation

Le projet est distribué sous la forme d'un dépôt Git. La première étape consiste donc à cloner le dépôt sur la machine locale :

```
1 git clone https://github.com/sayansuos/baggage-handling-rl.git
2 cd baggage-handling-rl
```

Il est ensuite recommandé de créer un environnement virtuel afin d'isoler les dépendances du projet. Celui-ci peut ensuite être activé selon le système d'exploitation utilisé :

- Sous macOS ou Linux :

```
1 python -m venv .venv
2 source ./venv/bin/activate
```

- Sous Windows :

```
1 python -m venv .venv
2 .\venv\Scripts\activate
```

Une fois l'environnement virtuel activé, les dépendances du projet peuvent être installées à partir du fichier `requirements.txt` :

```
1 pip install -r requirements.txt
```

Une fois cette étape terminée, le simulateur est prêt à être utilisé.

2 Structure du projet

2.1 Organisation des fichiers

Le projet est organisé en plusieurs dossiers, chacun regroupant un ensemble de fonctionnalités spécifiques. Le rôle des principaux fichiers et dossiers est résumé dans le Tableau 1.

TABLE 1 – Description des principaux éléments du projet.

<i>Élément</i>	<i>Description</i>
<code>configs/</code>	Fichiers de configuration du simulateur, regroupant les paramètres des environnements, des agents, des fonctions de récompense et des expériences d’entraînement.
<code>docs/</code>	Documentation du projet, comprenant le manuel d’utilisation, le document de modélisation ainsi que les comptes rendus du projet.
<code>figures/</code>	Figures, graphiques et animations générés lors de l’entraînement et de l’évaluation du modèle.
<code>logs/</code>	Journaux d’exécution et métriques générés lors de l’entraînement et de l’évaluation du modèle.
<code>rl/</code>	Implémentation des algorithmes d’apprentissage par renforcement ainsi que des réseaux de neurones associés.
<code>simulator/</code>	Cœur du simulateur. Ce dossier regroupe les principaux composants de la simulation (environnement, agents, obstacles), les outils nécessaires à l’apprentissage par renforcement (observations, fonction de récompense), la gestion des interactions entre les entités ainsi que le calcul des trajectoires des obstacles dynamiques par A^* .
<code>utils/</code>	Fonctions utilitaires utilisées par l’ensemble du projet (visualisation, journalisation, chargement des configurations, etc.).
<code>main.py</code>	Point d’entrée du projet, permettant de sélectionner le mode d’exécution du simulateur.
<code>runner.py</code>	Fonctions responsables du lancement des entraînements, des validations, des évaluations et de la génération des animations.

2.2 Architecture du simulateur

La Figure 1 présente l’architecture fonctionnelle du simulateur ainsi que les principales étapes d’exécution d’une expérience.

Quel que soit le mode d’exécution sélectionné (`train`, `validation`, `evaluation`, `animation` ou `demo`), le programme débute dans le fichier `main.py`. Celui-ci charge les fichiers de configuration afin d’initialiser l’environnement de simulation, les agents ainsi que les scénarios définis par l’utilisateur. Il sélectionne ensuite le mode d’exécution souhaité puis appelle les fonctions correspondantes du fichier `runner.py`.

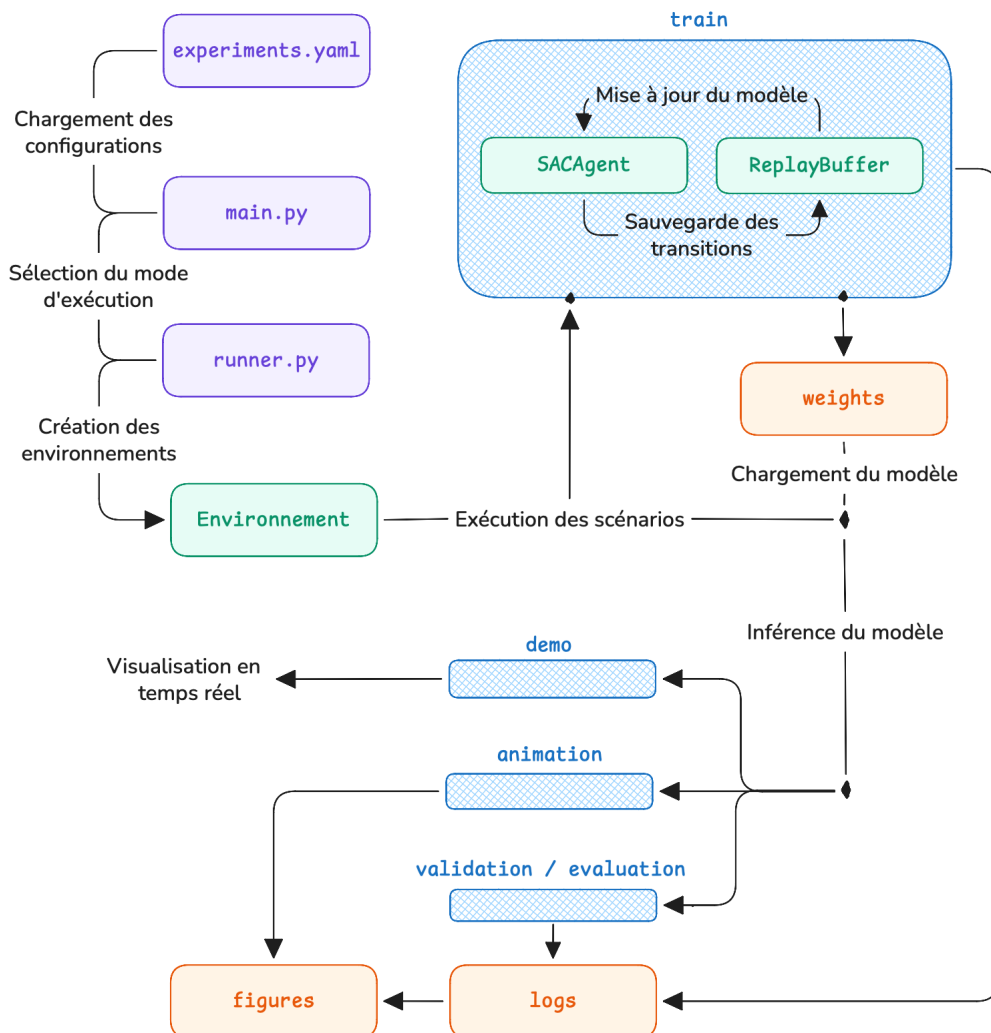
Une fois cette phase d’initialisation terminée, le comportement du simulateur dépend du mode

d'exécution choisi dans le runner.

En mode **train**, un agent est entraîné à l'aide de l'algorithme Soft Actor-Critic (SAC). À chaque interaction avec l'environnement, les observations sont utilisées pour sélectionner une action, qui est ensuite appliquée à l'environnement afin de produire un nouvel état et une récompense. Les transitions sont stockées dans le Replay Buffer et utilisées pour mettre à jour les réseaux de neurones. À la fin de l'entraînement, les poids du modèle ainsi que les différentes métriques sont enregistrés.

Les modes **validation**, **evaluation**, **animation** et **demo** utilisent quant à eux un modèle préalablement entraîné. Les poids sauvegardés sont chargés afin de calculer les actions de l'agent au cours de la simulation. Selon le mode sélectionné, le simulateur produit ensuite les métriques d'évaluation, les figures de résultats, une animation ou une démonstration en temps réel.

FIGURE 1 – Architecture fonctionnelle du simulateur



2.3 Organisation des classes

Le simulateur repose sur une architecture orientée objet dans laquelle les différents éléments présents dans un environnement sont représentés par des classes. La Figure 2 présente un diagramme de

classes simplifié des principaux composants du simulateur.

FIGURE 2 – Diagramme de classes simplifié du simulateur.

La classe `Entity` constitue la classe de base des objets présents dans l’environnement. Elle définit les propriétés communes aux différentes entités, notamment leur position, leurs dimensions ainsi que les mécanismes de détection des collisions.

Deux classes héritent de cette classe : la classe `StaticEntity` représente les obstacles statiques de l’environnement, tandis que la classe `MovingEntity` représente les entités capables de se déplacer. La classe `Agent` hérite elle-même de `MovingEntity` et complète son comportement avec les propriétés nécessaires à la navigation et à l’apprentissage par renforcement, telles que les commandes de vitesse, les observations locales et l’état de l’agent.

La classe `Environment` regroupe les agents, les obstacles statiques et les obstacles dynamiques composant un scénario. Elle est responsable de leur initialisation (via la classe `Manager`), de l’application des actions, de la mise à jour de la simulation, du calcul des observations et des récompenses, ainsi que de la détection de la fin d’un épisode.

La classe `GridMap` fournit une représentation discrète de l’environnement. Elle est utilisée à la fois pour construire les cartes locales observées par les agents et pour calculer les trajectoires des obstacles dynamiques à l’aide de la classe `AStar`.

Enfin, l’apprentissage par renforcement est assuré par la classe `SACAgent`, qui s’appuie sur un `ReplayBuffer` ainsi que sur les réseaux de neurones `FeaturesExtractor`, `ActorNetwork` et `CriticNetwork`. Les paramètres des environnements, des agents et de la fonction de récompense sont quant à eux définis respectivement par les classes `EnvironmentConfig`, `AgentConfig` et `RewardConfig`.

2.4 Principaux composants du simulateur

env – agent – obstacle rl – config – visu

3 Configuration du simulateur

Expliquer les fichiers de configuration

4 Utilisation du simulateur

Entrainement

Validation

Evaluation

Sauvegarde

5 Personnalisation du simulateur

créer un nouvel environnement

modifier la récompense

changer les observations

ajouter des obstacles

modifier les agents

6 Résultats générés

logs/

weights/

figures/

Références